Failure (a real rollback trigger) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `rollback_trigger_practice.py`: a rollback plan that has never actually fired is still just an idea on paper. This file plays back a real score sequence that genuinely drops below threshold, and checks that the trigger fires at exactly the right moment — and only that once, not repeatedly. **If not:** the first time your rollback trigger ever ran for real would be during an actual production incident.

Every example below plays back this same simulated score sequence — a healthy start, then a sustained drop partway through, acting out the exact situation this exercise asks for:

```
# rollback_trigger_practice.py
scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
baseline = 0.95
```

Solution

Story: the terminal should stay reasonably quiet during normal operation, but after a real incident you want every single window check that ran recorded somewhere. Two separate log levels — `INFO` printed to screen, `DEBUG` written to a file — give you both at once. **If not:** "how fast did the score actually fall?" would have no answer at all after the fact, with nothing recorded to check.

```
# rollback_trigger_practice.py
import logging

logger = logging.getLogger("rollback")
logger.setLevel(logging.DEBUG)
console = logging.StreamHandler()
console.setLevel(logging.INFO)
logfile = logging.FileHandler("rollback.log")
logfile.setLevel(logging.DEBUG)
logger.addHandler(console)
logger.addHandler(logfile)

def should_rollback(
    scores_so_far: list[float],
    baseline: float,
```

```
    window_size: int = 5,
    drop_threshold: float = 0.2,
) -> bool:
    # when: not enough scores yet for a fair average — never fire early
    if len(scores_so_far) < window_size:
        return False
    # how: only the most recent window counts
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    logger.debug("window avg=%.3f drop=%.1f%%", avg_recent, percent_drop * 100)
    return percent_drop >= drop_threshold

def rollback_to_previous() -> str:
    logger.critical("ROLLBACK: reverting to last known-good version")
    return "v1"

def watch_and_rollback(scores: list[float], baseline: float) -> None:
    seen: list[float] = []
    for i, s in enumerate(scores):
        seen.append(s)
        if should_rollback(seen, baseline):
            print(f"request {i}: score {s} -- triggering rollback")
            rollback_to_previous()

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
watch_and_rollback(scores, baseline=0.95)
```

Expected output: the same firing pattern — fires at request 7, and again at 8 and 9, since `should_rollback` still has no memory between calls. The terminal shows only the `request N` lines and the `ROLLBACK` messages; `rollback.log` *also* holds a `DEBUG` line for every window check, like `window avg=0.722 drop=24.0%`.

Revision from Doc01 (Intermediate): this is Doc01's "logger with two output levels" exercise, reused as-is: the terminal stays quiet (`INFO` and above), while the file keeps every detail (`DEBUG` and above). After a real rollback, the first question is always "how fast did the score fall?" — the `DEBUG` lines in `rollback.log` answer it, without filling the terminal during normal operation.